

Deriving sequential programs from Event-B models

- Combining program and data refinement

Michael Butler, Sadegh Dalvandi, Abdolbaghi Rezazadeh

WG 2.3 Meeting 58
Villebrumier, October 2016

Specification of search

variable items $\in 0..N-1 \rightarrow T$

event FindX

any i

where

$i \in 0..N-1$

items(i) = X

then

result := i

end

event NoFind

when

$X \notin \text{ran}(\text{items})$

then

result := -1

end

Event-B refinement

- one-to-many event refinement

$$e \sqsubseteq_I e', \quad e \sqsubseteq_I e'' \quad (I \text{ is an invariant})$$

- new events (stuttering steps)

$$\text{skip} \sqsubseteq_I f$$

f decreases variant *(f is convergent)*

- Proof method:

- invariants + variant
- pre-defined proof obligations

Refinement of search (linear)

variable $k \in 0..N$

invariant $X \notin \text{ran}(\text{items}[0..k-1])$

variant $N-k$

init $k := 0$

event Step

when

$k < N$

$\text{items}(k) \neq X$

then

$k := k+1$

end

event FindX **refines** FindX

when

$k < N$

$\text{items}(k) = X$

then

$\text{result} := k$

end

event NoFind **refines** NoFind

when

$k = N$

then

$\text{result} := -1$

end

Implementation of search

Ultimately we could derive this code:

init ;

while $k < N \wedge a(k) \neq X$ **do** $k := k + 1$ **end** ;

if $k \leq N$ **then** $\text{result} := k$ **else** $\text{result} := -1$ **end**

Event schedule

init ; (FindX $\square$ NoFind)

$\leadsto$

init ; Step* ; (FindX $\square$ NoFind)

$\leadsto$

init ;

while $k < N \wedge a(k) \neq X$ **do** $k := k + 1$ **end** ;

if $k \leq N$ **then** result:=k **else** result:=-1 **end**

Event schedule language

- Regular expression constraining event ordering
- Syntax:

$$\begin{array}{l} S ::= \text{EventName} \\ \quad | S ; S \\ \quad | S \square S \\ \quad | S^* \end{array}$$

Refinement rule

- Loop intro:

schedule: $e \leadsto f^* ; e'$

events: $\text{skip} \sqsubseteq f, \quad e \sqsubseteq e'$

convergent: f

Event groups

- Event group:

$$E = (e_1 \sqcup e_2 \sqcup \dots \sqcup e_n)$$

- $E \sqsubseteq E'$ is short for $e_i \sqsubseteq e'_i$ for each i
- $e \sqsubseteq E'$ is short for $e \sqsubseteq e'_i$ for each i

Loop rules

- Loop intro:

schedule: $E \leadsto F^* ; E'$

event: $\text{skip} \sqsubseteq F, \quad E \sqsubseteq E'$

conv: F

- Loop extension:

schedule: $E^* \leadsto (F \sqcap E')^*$

event: $\text{skip} \sqsubseteq F, \quad E \sqsubseteq E'$

conv: F

Case split rule

- Case split :

schedule: $e \rightsquigarrow E'$

event: $e \sqsubseteq E'$

Binary search

init ; (FindX $\square$ NoFind)

$\leadsto$ { loop intro }

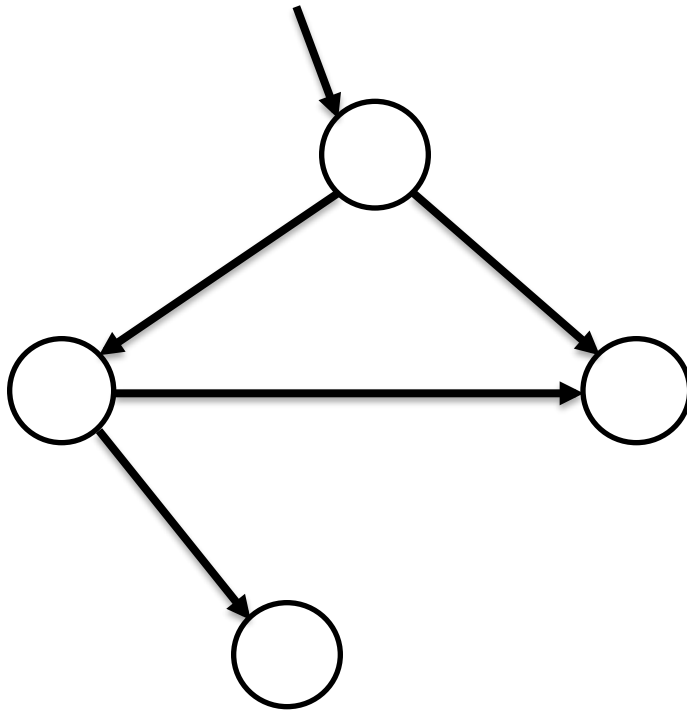
init ; Step* ; (FindX $\square$ NoFind)

$\leadsto$ { case split, data refinement }

init ; (Step_Up $\square$ Step_Down)* ; (FindX $\square$ NoFind)

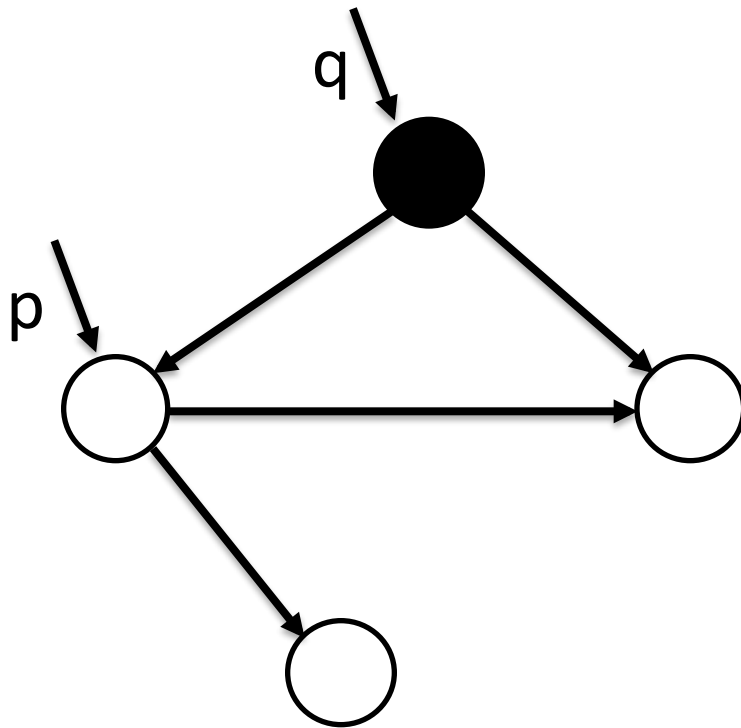
Schorr-Waite

- Mark all reachable nodes in graph



Schorr-Waite

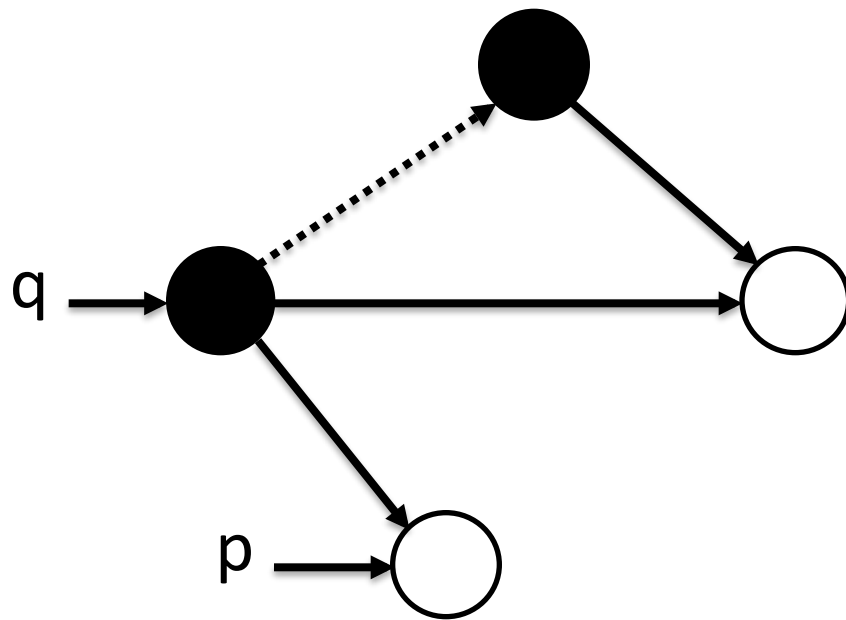
- Mark all reachable nodes in graph



Move ahead

Schorr-Waite

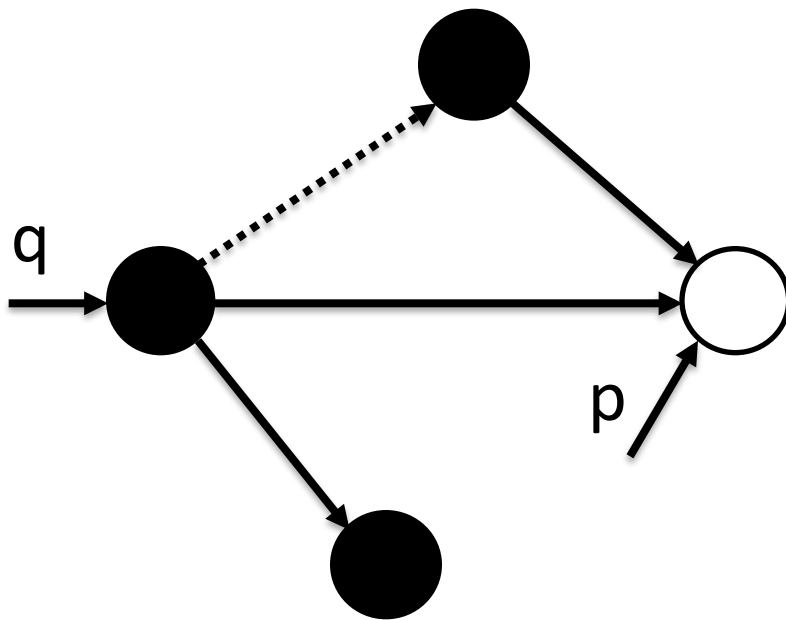
- Mark all reachable nodes in graph



Move ahead

Schorr-Waite

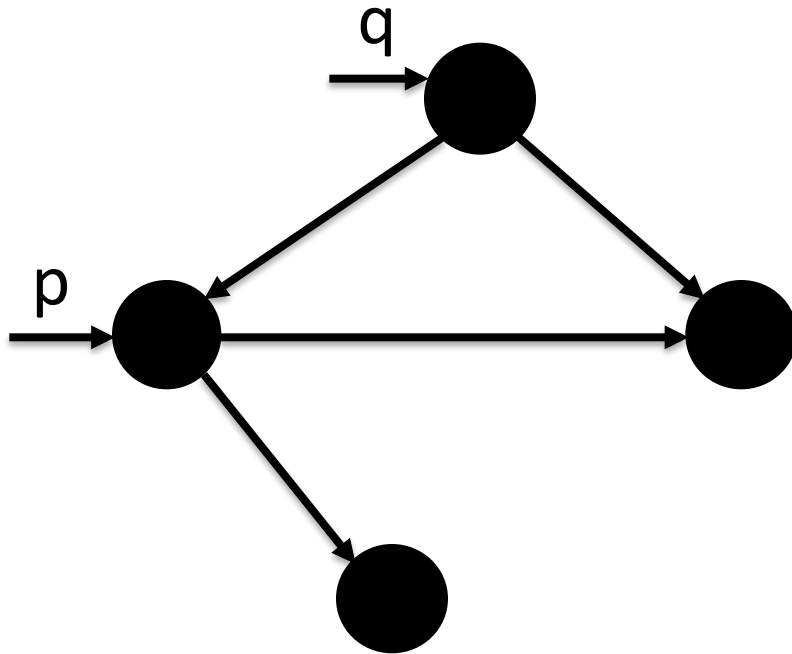
- Mark all reachable nodes in graph



Move across

Schorr-Waite

- Mark all reachable nodes in graph



Backtrack

Schorr-Waite schedule refinement

init ; Marked

→ { loop intro }

init ; Mark* ; Marked

→ { loop extend }

... ; (Backtrack □ Mark)* ; ...

→ { loop extend }

... ; (NextChild □ Backtrack □ Mark)* ; ...

Schorr-Waite data refinement

init ; Marked // $g \in N \leftrightarrow N$

~ { loop intro }

init ; Mark* ; Marked // $\text{marked} \subseteq N$

~ { loop extend }

... ; (Backtrack $\square$ Mark)* ; ... // $b \in N \rightsquigarrow N$

~ { no change }

... ; (Backtrack $\square$ Mark)* ; ... // $\text{marked2} \in N \rightarrow \text{BOOL}$

~ { loop extend }

... ; (NextChild $\square$ Backtrack $\square$ Mark)* ; ... // assume order on children

~ { no change }

... ; (NextChild $\square$ Backtrack $\square$ Mark)* ; ... // $\text{cg} \in N \leftrightarrow N$

If and While intro

- While intro:

$E^* \leadsto \text{while } G \text{ do } E \text{ od}$

- If intro

$E \sqcap F \leadsto \text{if } G \text{ then } E \text{ else } F \text{ end}$

$E \leadsto \text{if } G_1 \text{ then } e_1$
 $\quad \text{elseif } G_2 \text{ then } e_2 \dots$
 $\quad \text{else } e_n \text{ end}$

What to do with event guards?

init ; (FindX $\square$ NoFind)

↷

init ; Step* ; (FindX $\square$ NoFind)

↷

init ;

while $k < N \wedge a(k) \neq X$ **do** Step **end** ;

if $k \leq N$ **then** FindX **else** NoFind **end**

Event specs

```
event Step refines skip
  when
     $k < N$ 
     $\text{items}(k) \neq X$ 
  then
     $k := k + 1$ 
end
```

```
event FindX refines FindX
  when
     $k < N$ 
     $\text{items}(k) = X$ 
  then
     $\text{result} := k$ 
end
```

```
event NoFind refines NoFind
  when
     $k = N$ 
  then
     $\text{result} := -1$ 
end
```

Eliminating event guards

init ; (FindX $\square$ NoFind)

$\leadsto$

init ; Step* ; (FindX $\square$ NoFind)

$\leadsto$

init ;

while $k < N \wedge a(k) \neq X$ **do** Step **end** ;

if $k \leq N$ **then** FindX **else** NoFind **end**

$\leadsto$

init ;

while $k < N \wedge a(k) \neq X$ **do** $k := k+1$ **end** ;

if $k \leq N$ **then** result:=k **else** result:=k **end**

Propagation of schedule guards

if G then E else F end

$\leadsto$ **if G then {G}E else {¬G}F end**

while G do E end

$\leadsto$ **while G do {G}E end {¬G}**

$\{G\} E \leadsto E \{G\}$, provided E maintains G

$\{G\} E^* \leadsto E^* \{G\}$, provided E maintains G

Enabledness preservation

- Loop intro:

schedule: $E \leadsto F^* ; E'$

event: $\text{skip} \sqsubseteq F, \quad E \sqsubseteq E'$

conv: F

enable: $I \wedge \text{gd}(E) \Rightarrow \text{gd}(F) \vee \text{gd}(E')$

- Loop extension:

schedule: $E^* \leadsto (F \sqcap E')^*$

event: $\text{skip} \sqsubseteq F, \quad E \sqsubseteq E'$

conv: F

enable: $I \wedge \text{gd}(E) \Rightarrow \text{gd}(F) \vee \text{gd}(E')$

Guards and preconditions

- Strengthen guard:

$$g \rightarrow A \sqsubseteq g \wedge h \rightarrow A$$

- Weaken precondition (refinement calculus)

$$[P, Q] \sqsubseteq [P \vee R, Q]$$

Example: case split

- Event case split

$$g \rightarrow A \sqsubseteq_I g1 \rightarrow A1$$

$$g \rightarrow A \sqsubseteq_I g2 \rightarrow A2$$

- Refinement calculus IF-intro

$$[P,Q] \sqsubseteq \text{if } G \text{ then } [G \wedge P,Q] \text{ else } [\neg G \wedge P,Q] \text{ end}$$

but

$$[P,Q] \not\sqsubseteq [G \wedge P,Q]$$

$$[P,Q] \not\sqsubseteq [\neg G \wedge P,Q]$$

Data refinement of programs in refinement calculus

- $D(S)$ is least data refinement of program S

- Laws

$$D(S ; T) \sqsubseteq D(S) ; D(T)$$

$$D(\text{if } G \text{ then } S \text{ else } T \text{ end})$$

$$\sqsubseteq \text{if } D(G) \text{ then } D(S) \text{ else } D(T) \text{ end}$$

$$D(\text{while } G \text{ do } S \text{ end})$$

$$\sqsubseteq \text{while } D(G) \text{ do } D(S) \text{ end}$$

Method I

- Using *abstract* schedules ($\square ; *$)
 - record intended program refinement steps
 - but they don't change behaviour of Event-B model
 - data refinement verification is Event-B
 - plus enabledness preservation
 - observation: minimal use of ;

Method II

- Produce *concrete* ‘code’ (after data refinement)
 - replace $\square *$ with IF, WHILE
 - concrete schedule constrains order of events
 - so is a correct refinement of Event-B model
 - elimination of event guards leads to proof obligations
 - enabled preservation helps ensure event guards are not too strong to be eliminated

Refinement rule

- Seq intro:

schedule: $e \leadsto f ; e'$

events: $\text{skip} \sqsubseteq f, \quad e \sqsubseteq e'$

convergent: f

enabledness: $f \{ \text{grd}(e') \} \quad (f \text{ enables } e')$

END